Bridging Keyword PIR and Index PIR via Minimal Perfect Hash Function and Batch PIR

Huiqiang Liang
Harbin Institute of Technology
24b903126@stu.hit.edu.cn

Haining Yu
Harbin Institute of Technology
yuhaining@hit.edu.cn

Changtong Xu
Independent Researcher
changtong1996@gmail.com

Dongyang Zhan
Harbin Institute of Technology
zhandy@hit.edu.cn

Jinbo Yang
Harbin Institute of Technology
24b903125@stu.hit.edu.cn

Hongli Zhang
Harbin Institute of Technology
zhanghongli@hit.edu.cn

Abstract

This paper presents a Keyword Private Information Retrieval (Keyword PIR) scheme that achieves a constant-factor online computation and communication overhead compared to the underlying Index PIR, bridging the gap between Keyword PIR and Index PIR, and enabling efficient and privacy-preserving queries over diverse databases. We introduce a new Key-Value Store (KVS) instantiate by Minimal Perfect Hash Function, referred to as MPHF-KVS, in which each keyword query requires only a single index query. We then develop a generic Batch PIR framework that converts Index PIR into Keyword PIR using KVS encoding. In particular, when the KVS is instantiated using a Binary Fuse Filter (BFF-KVS), Keyword PIR can be reduced to Batch PIR. Leveraging the updatable hint structure of PIR with side information, we propose a novel Rewind & Skip technique that enables the execution of multiple queries within a single round.

In MPHF-KVS, the online computation and communication costs are at most $2\times$ those of Index PIR. In our Batch PIR with BFF-KVS, building upon three recent PIR schemes with sublinear server-side online computation and communication cost and without extra hint store, our approach inherits their advantages and achieves keyword query costs of less than $7\times$ the cost of an index query, while still maintaining sublinear online complexity.

1 Introduction

Private Information Retrieval (PIR) [10] allows a client to retrieve an element from a server's database without revealing which element was queried. Due to its strong guarantees of client privacy, PIR has become a fundamental building block in many applications, including password breach checking [2,43], privacy-preserving advertising [20], private contact discovery [23], and certificate transparency auditing [21]. A long line of research [1–15, 21, 22, 28, 31–39, 42, 44–46] has focused on making PIR more practical by developing simpler architectures and achieving better trade-offs between communication and computation costs.

Considering a database $\mathsf{DB} = \{\alpha_0, \ldots, \alpha_{n-1}\}$, a straightforward way to implement PIR is for the client to download the entire database locally, which incurs $O(n)$ communication and $O(n)$ client storage. A more advanced approach typically involves the client using a homomorphic encryption to encrypt the query index as a one-hot vector [8]. The server then evaluates this encrypted query over the entire database, resulting in a PIR scheme with $O(n)$ online communication and server-side computation. To reduce the communication overhead, many scheme [8, 21, 28] partition the query index into a matrix or cube structure, reducing the online communication complexity to $O(\sqrt{n})$ or $O(\sqrt[3]{n})$, at the cost of slightly increased server-side computation due to additional multiplication operations. Furthermore, the high computational overhead of homomorphic encryption and its large ciphertext expansion ratio impose significant bandwidth and limited the performance, leaving substantial room for improving the overall communication-computation trade-off [7, 36].

When the database is public, a compromise approach allows the client to compute side information representing the partial sums of subsets during the offline phase, and then, in the online phase, retrieve all but one element from the subset to reconstruct the desired value. This method [12] achieves a better trade-off between offline storage and server-side online computation. In the theoretical analysis, when no side information is available, the server must access every bit of the database to answer a client query [5]; otherwise, it could immediately infer which bits are outside the client's query scope. There is a fundamental lower bound of $S \cdot T \geq \Omega(n)$ [11, 12], where S denotes the client's storage and T the server's computational cost. Thus, a key advantage of PIR schemes leveraging side information [4, 12, 38, 44, 46] is that the server can reduce its online computation cost from linear to sublinear, significantly enhancing its capability to handle large-scale databases.

In most real-world applications, databases are stored as key-value pairs $\mathsf{KV} = ((k_0, v_0), \ldots, (k_{m-1}, v_{m-1}))$, and clients retrieve values using the corresponding keys. Compare to the traditional Index PIR, the main difference in Keyword

PIR [10] is that clients query based on keys rather than numerical indices. Keyword PIR can be built upon Index PIR. A straightforward approach is to translate keyword queries into index queries by arranging key-value pairs sequentially and mapping keys to index vectors. However, this approach requires the client to maintain the key-to-index mapping locally. Alternatively, the client can send an encrypted keyword, and the server performs equality comparisons [32] to return the corresponding value, but this approach significantly increases computational overhead. Another promising approach is the use of Key–Value Stores (KVS) [3, 9, 34], implemented using Cuckoo hashing [17] or Binary Fuse Filters (BFF) [19], which encode key–value pairs into an array-like structure and then apply Index PIR. This method requires querying multiple indices to retrieve a single keyword, and thus the efficiency of these combined methods remains limited.

Our works. Current state-of-the-art Keyword PIR schemes [2, 9, 32, 34, 39] can classify to an Index PIR with three encoding methods: key-to-index mapping, equality comparisons and KVS. The underlying PIR based on homomorphic encryption (HE-PIR) and side information (SI-PIR). However, key-to-index mapping approaches often require the client to store a large index structure, while BFF-KVS methods usually involve multiple index queries per keyword, which significantly limits scalability. To address these issues, we instantiate the KVS using a Minimal Perfect Hash Function (MPHF) [27] to record the position of each key in the key set. The MPHF requires less than 1.444 bits per key, thereby dramatically reducing the client-side storage overhead, and enables each keyword query to be equivalent to a single index query. A key challenge in using MPHF-KVS is verifying whether a queried key exists in the key set. To address this, we append a fingerprint of each key to its associated value and employ a slow hash function to resist brute-force attacks from the fingerprint to the original key..

In addition, the MPHF-KVS is directly applied to convert Index PIR into Keyword PIR, the KPIR [34] use the BFF-KVS and HE-PIR. So we make the Keyword PIR compatible with SI-PIR [4, 44, 46] under the BFF-KVS encoding method. By encoding the key-value pairs into an indexable array, querying a key becomes equivalent to querying multiple random indices, effectively reducing Keyword PIR to Batch PIR. A key challenge in Batch PIR under the SI-PIR (Batch SI-PIR) is that it requires refreshing the client's storaged hints after each server response, which incurs multiple rounds of interaction. We propose a novel Rewind & Skip technique to synchronize hints updates across multiple queries, enabling correct and efficient batching. Finally, our MPHF-KVS and Batch PIR bridges the gap between Keyword PIR and Index PIR, introducing only a small constant-factor overhead in computation and communication compared to the original Index PIR schemes.

The contributions. In this work, we achieve the following.

- We propose the MPHE-KVS, use the Minimal Perfect Hash Function (MPHF) [27] to implement the KVS encoding, add a fingerprint function to configurable false-positive.
- We present a general Batch PIR framework and utilize SI-PIR [4, 44, 46] to realize its Batch SI-PIR, which is then extended to Keyword PIR via BFF-KVS encoding. A new Rewind & Skip technique allows the client to execute multiple random queries in a single round.
- In trem of performance, under the MPHF-KVS encoding, Each keyword query no more than $2\times$ computation and communications overhead to index query. Under the BFF-KVS encoding method, Each keyword query incurs on more than $7\times$ computation and communications overhead compared to a single index query while preserving the single interactive round.

1.1 Overview of Our Techniques

Our approach is straightforward: (1) encode key-value pairs into a compact, indexable array; (2) make Index PIR batchable for multiple random queries; and (3) use the batched queries retrieve multiple entries that reconstruct the value of a given key. For clarity, given a key-value map KV, we construct an indexable array DB using a sparse Key-Value Store (KVS) [34] implemented with the BFF algorithm [19], which we refer to as BFF-KVS. The value $\mathsf{KV}[k]$ is reconstructed by accessing the entries $\mathsf{DB}[h_i(k)]_{i\in[t]}$, where each h_i is a hash function mapping key to an index. These entries satisfy $\oplus_{i\in[t]}\mathsf{DB}[h_i(k)] = \mathsf{fpt}_\varepsilon(k,v)$, where

$$\mathsf{fpt}_\varepsilon(k,v) = \mathsf{hash}(k)\|v \tag{1}$$

enabling us to recover $v = \mathsf{KV}[k]$ from $\mathsf{DB}[h_i(k)]_{i\in[t]}$. This construction is used in ChalametPIR [9] and SparsePIR [39]. The BFF-KVS embedd the corresponding value v after $\mathsf{hash}(k)$, introducing extra fingerprint information in each slot and a certain false-positive rate, to transform the key-value pairs into an indexable array.

MPHF-KVS. Next, we observe that BFF-KVS requires 3 or 4 index queries per keyword, whereas the key-to-index mapping approach needs only one index query but incurs a large client-side memory, therefore, we aim to compress this extra memory. In 1997, Chor et al. [6] introduced the concept of a perfect hash function, which maps a keys set $S = \{k_0,\ldots,k_{m-1}\}$ to the range $[0,\ldots,m-1]$ without collisions. With an appropriate arrangement of the keys, we can obtain

$$\mathsf{MPHF}(k'_i) = i \tag{2}$$

where $\{k'_0,\ldots,k'_{m-1}\}$ and S are same elements but differing in their ordering. At that time, no practical algorithm was available for efficiently constructing such perfect hash functions. Recent advances [16, 25, 27, 29, 41] have made them highly

efficient, requiring as little as 1.444 bits per key. This dramatically reduces the extra client-side memory while allowing only a single index query per keyword lookup.

For queries on keys outside the original key set, an MPHF returns an arbitrary index, which may leak random values. To address this issue, we define $\mathsf{DB}[i] = \mathsf{fpt}_\varepsilon(k'_i, v'_i)$ and

$$\mathsf{fpt}_\varepsilon(k, v) = \mathsf{hash}_1(k) \| (\mathsf{hash}_2(k) \oplus v) \tag{3}$$

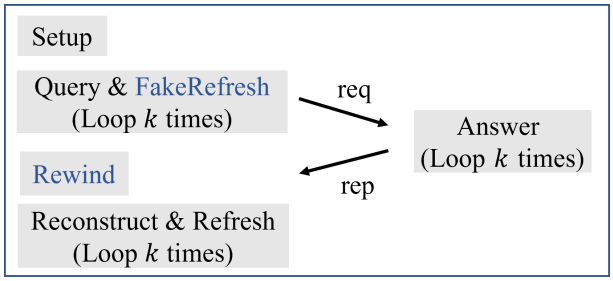
Here, $\mathsf{hash}_1(\cdot)$ is used to verify whether the queried key exists, while $\mathsf{hash}_2(\cdot)$ ensures that the output appears random.

This completes the construction of our MPHF-KVS. Compared to Index PIR, the additional cost of Keyword PIR under MPHF-KVS primarily consists of extra memory of each key, as well as the encoding/decoding time and additional fingerprint storage in the database. Apart from these, the computational and communication costs for keyword queries remain identical to those for index queries.

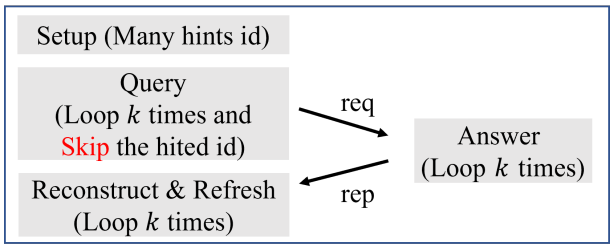
Batch SI-PIR. In BFF-KVS encoding, the client retrieves entries at multiple random indices. One approach issues separate queries for each index, but this introduces multiple rounds and increases network latency. To avoid this, we design a Batch SI-PIR scheme that allows querying multiple random indices in a single round. Unlike the HE-PIR, which does not require hint updates after each query and can directly support one-round batch version. The SI-PIR [4, 44, 46] involves a refresh phase that updates the client's hints after every server response. The key insight is that client hints consist of two parts, the index sets and their corresponding partial sums. A query contains only an index set, and the server response returns the corresponding values or partial sums. During the refresh phase, the response is used to update the partial sums for a new index set. Importantly, the new index set can be generated independently of the server response, and the corresponding partial sums can be updated upon receiving the response. This enables single-round Batch SI-PIR.

The main challenge is synchronizing updates to the client's hints. We address this using a Rewind technique: the client saves its current hints and the nonce before sending the query, then updates its hints based on the nonce. After receiving the response, the client rewinds to the saved hints, replays the nonce and updates its hints using the response, thus simulating the normal sequential process. The Skip technique is general, it replicates several copies of the client's hint, thereby enabling Batch SI-PIR in a single round at the cost of additional client storage. The key difference from Piano [46] is that it stores many hints consisting of multiple index sets and partial sums. Each query index hits a particular hint, which is refreshed after the response. Thus, we can apply the Skip technique directly: if a particular hint is hit, we skip it and search for another. These two methods are illustrated in Figure 1.

As a result, the cost of the Rewind & Skip techniques is negligible, involving only a few assignment and comparison operations, making the total cost of Batch SI-PIR comparable to that of the original PIR. While the KVS produce 3 indices per key, with an encoded database size of $n < 1.15m$, and additional 1-byte fingerprint context, each keyword query in our Keyword PIR incurs less than $7\times$ the online costs compared to the original schemes.



(a) Rewind technique



(b) Skip technique

Figure 1: The Rewind & Skip Techniques. In a two-party setting, the client runs Setup and then Query, sending req to the server. The server executes Answer and returns rep, and the client uses Reconstruct & Refresh to get the result and update hints. In (a), FakeRefresh refreshes hints without the answer; the client generates a batch req, rewinds hints upon receiving rep, then Reconstruct & Refresh to synchronize. In (b), with multiple hint copies, the client can select different id in a batch query and refresh hints after receiving the rep.

1.2 Related Works

We begin by revisiting existing PIR schemes, which are primarily based on two underlying techniques: homomorphic encryption and side information. We then introduce the concept of Keyword PIR.

PIR based on Homomorphic Encryption (HE-PIR). SimplePIR [21] is a representative scheme based on the Learning With Errors (LWE) assumption, where the ciphertext is of the form $(A, b = A \cdot s + \Delta \cdot m + e)$. The database is encoded as a matrix $\mathsf{DB} \in \mathbb{Z}^{\sqrt{n} \times \sqrt{n}}$, and m is a one-hot vector, then the server computes $(\mathsf{DB} \cdot A, \mathsf{DB} \cdot b)$ to retrieve $\mathsf{DB} \cdot m$, i.e., a column of the database. DoublePIR was later proposed to support retrieval of single elements. To reduce online computation, $\mathsf{DB} \cdot A$ is precomputed as a client-side hint. A critical aspect is parameter selection, the server performs only 32-bit additions and multiplications on each element of DB to complete the PIR, pushing performance to its practical limits.

After that, HintlessPIR [28] removes the client hints used in SimplePIR [21] by encrypting the secret vector s, enabling the server to compute $\mathsf{DB} \cdot A \cdot s$. It also proposes TensorPIR, a three-dimensional scheme that reduces communication and supports offline/online precomputation. YPIR [36] based on Ring LWE, replaces the matrix A in SimplePIR [21] with a cyclic matrix, significantly reducing hint size via packing techniques. It eliminates offline communication and decreases response size, but increase the client query size. Later, InsPIRe [33] using a new ring packing algorithm, compare to YPIR [36], it reduce $5\times$ cryptographic keys and half of query size. These PIR schemes using with homomorphic encryption do not require a trusted setup during the offline phase; however, they incur linear server online computation, making them inefficient for large datasets.

PIR based on Side Information (SI-PIR). Beimel et al. [5] demonstrated that by precomputing and storing the XOR results of database subsets, servers can reduce the amount of computation required for each query. Patel et al. [38] proposed a private stateful information retrieval scheme in which, following an offline phase, the server can perform sublinear computation for each query. Corrigan-Gibbs and Kogan [12] introduced a PIR scheme with distinct online and offline phases. After an offline phase of $\tilde{O}(n)$ time, it achieving an optimal trade-off between communication and online time characterized by $S \cdot T \geq \tilde{\Omega}(n)$ where $\tilde{\Omega}(\cdot)$ hides polylogarithmic factors.

More practical work by Piano [46] requires $\tilde{O}_\lambda(\sqrt{n})$ client storage and achieves $\tilde{O}(\sqrt{n})$ computation for both server and client, with $O(\sqrt{n})$ online communication per query. In the single-server setting, the online phase is fast, allowing multiple queries to be issued, though not unlimited, while the offline hint construction involves a relatively lengthy process. Checklist [14] is a two-server PIR scheme for private blocklist lookups, where one server handles queries and the other manages refresh operations. Its security relies on the two servers non-collude assumption. Lazzaretti and Papamanthou [4] improved Checklist [14] by introducing a "single-pass" client offline phase, which reduces preprocessing time. They also enabled faster queries and refresh operations at the cost of increased answer communication. Wang and Ren [44] removed the dedicated refresh server from [4] by introducing dummy placeholders, thus reducing query and answer communication by half, which matches the lower bound $ST = \Omega(nw)$ in [45], up to constant factors. These SI-PIR schemes require a trusted setup during the offline phase to access the database for hint construction. With the help of these hints, they achieve sublinear server computation.

Keyword PIR. Query from key instead of index. Chor et al. [6] first propose a Keyword PIR scheme that requires logarithmic rounds of HE-PIR over a binary tree structure, the trie and perfect hash function. Ali et al. [2] design a single-round approach by leveraging standard or Cuckoo hashing in combination with Index PIR. Mahdavi and Kerschbaum [32] present an equality-testing method based on constant-weight encoding, though it introduces substantial computational overhead. Checklist [14] stores the key-to-index mapping locally on the client side, trading increased client storage for transforming keyword queries into index queries. An important insight is that, using the key-to-index mapping, all Keyword PIR schemes can be transformed into Index PIR.

For improved efficiency. SparsePIR [39] proposes a novel partitioning algorithm that integrates with underlying Index PIR schemes [35, 37]. ChalametPIR [9] combines Binary Fuse Filter (BFF) [19] with Index PIR (e.g., SimplePIR [21] or FrodoPIR [1]) to enable keyword queries. Recently, KPIR [34] generalizes SimplePIR [21] into a Row-KOPIR abstraction. It further extends BFF into sparse-KVS to represent key-value pairs as an arrays, and explores additional encoding strategies such as hashing-to-bins and approximate key-to-index mapping. However, all of these schemes are based on HE-PIR.

Others. Marian et al. [13] address malicious clients by incorporating authentication steps and requiring multiple queries to ensure integrity and soundness. Adria [18] introduces a shuffle-based PIR model that relies on multiple non-colluding servers, enhancing privacy through secure data permutation. The Distributional PIR [42], proposed by Ryan Lehmkuhl et al., focuses on the distribution of the database by maintaining a small "hot area" to store frequently queried items alongside the original database. Queries are more likely to hit this hot area, improving performance. Importantly, this method can be applied to all existing PIR schemes if a small false probability is acceptable, which can be further reduced by repeating queries.

2 Preliminaries

We use κ and λ to denote the computational and statistical security parameters. The notation $[n]$ represents the set $\{0, 1, \ldots, n-1\}$, and $\|$ denotes concatenation. The $\mathsf{negl}(\cdot)$ denote the negligible function. The PRP denotes a pseudorandom permutation, and the PRF denotes a pseudorandom function.

2.1 Keyword Private Information Retrieval

Similar to prior works [1, 21, 28, 34], we assume that Keyword PIR includes a query-independent setup phase, during which the client downloads the entire database from the server, generates a local hint, and subsequently deletes the database.

A Keyword PIR consisting of key-value pairs $\mathsf{KV} = (k_i, v_i)_{i\in[m]} \in (\mathcal{K} \times \mathcal{V})^m$, following the four algorithms:

- $\mathsf{Setup}(1^\lambda, 1^\kappa, \mathsf{KV}) \to (\mathsf{DB}, \mathsf{hint}_s, \mathsf{hint}_c)$: Given the database KV, outputs the encode database DB, the server-side hint_s and the client-side hint_c.

- $\mathsf{Query}(\mathsf{hint}_c, k) \to (\mathsf{st}, q)$: Given a key $k \in \mathcal{K}$ and the client hint_c, outputs a client state st and a query q to be sent to the server.

- $\mathsf{Answer}(\mathsf{hint}_\mathsf{s}, \mathsf{DB}, q) \to a$: Upon receiving the query q, and given the server-side hint_s and the encoded database DB, outputs an answer a.

- $\mathsf{Reconstruct}(\mathsf{hint}_c, \mathsf{st}, a) \to v$: Upon receiving the answer a, with the client's hint_c and state st, output a value $v \in \mathcal{V}$. The hint and state are then updated as follows: $(\mathsf{hint}_s, \mathsf{hint}_c) \leftarrow \mathsf{Refresh}(\mathsf{hint}_s, \mathsf{hint}_c, \mathsf{st}, a)$ where $\mathsf{Refresh}$ is a deterministic function.

It should satisfy two properties: correctness and security.

Definition 2.1. (Correctness) A Keyword PIR is correct if, for any database KV and any key $k_i \in \mathsf{KV}$, the following holds:

$\mathsf{Reconstruct}(\mathsf{hint}_\mathsf{c}, \mathsf{st}, \mathsf{Answer}(\mathsf{hint}_\mathsf{s}, \mathsf{DB}, q)) = \mathsf{KV}[k]$

where $(\mathsf{DB}, \mathsf{hint}_\mathsf{s}, \mathsf{hint}_c) \leftarrow \mathsf{Setup}(1^\lambda, 1^\kappa, \mathsf{KV})$ and $(\mathsf{st}, q) \leftarrow \mathsf{Query}(\mathsf{hint}_c, k)$ with probability at least $1 - \mathsf{negl}(\lambda) - \mathsf{negl}(\kappa)$. If the queried key $k \notin \mathsf{KV}$, then the $\mathsf{Reconstruct}$ outputs $\perp$ with probability $1 - \mathsf{negl}(\kappa)$.

Definition 2.2. (Correctness up to L queries) A Keyword PIR is said to be correctness up to L queries, if it guarantees correctness for at most L queries, and outputs $\perp$ with probability $1 - \mathsf{negl}(\kappa)$ once the queries exceeds L.

Definition 2.3. (Security) A keywordPIR is secure if, for any probabilistic polynomial time adversary $\mathcal{A}$ and for any pair of keys $(k_i, k_j) \in \mathcal{K}$, the following holds:

$$\left|\Pr[\mathcal{A}(q) = 1 \mid (\mathsf{st}, q) \leftarrow \mathsf{Query}(\mathsf{hint}_c, k_i)] - \Pr[\mathcal{A}(q) = 1 \mid (\mathsf{st}, q) \leftarrow \mathsf{Query}(\mathsf{hint}_c, k_j)]\right| \leq \mathsf{negl}(\kappa) \quad (4)$$

2.2 Sparse Key-Value Store

A Key-Value Store (KVS) [34] is an encoding method that maps key-value pairs to an indexable array. A γ-sparse KVS maps a collection of key-value pairs $\mathsf{KV} = \{(k_i, v_i)\}_{i \in [m]} \in (\mathcal{K} \times \mathcal{V})^m$ into a vector $\mathsf{DB} \in \mathcal{Z}^n$, using the following three algorithms:

- $\mathsf{KVS.Setup}(\varepsilon, \omega, m) \to \mathsf{pp}$: Given the false-positive probability ε and size m, the radio ω about $\mathsf{hint}_\mathsf{kvs}$, output the public parameters pp.

- $\mathsf{KVS.Encode}(\mathsf{pp}, \mathsf{KV}) \to (\mathcal{H}, \mathsf{hint}_\mathsf{kvs}, \mathsf{DB})$: Given the parameters pp and a set of key-value pairs KV, outputs an array $\mathsf{DB} \in \mathcal{Z}^n$, a collection of hash functions $\mathcal{H} = \{h_i\}_{i \in [\gamma]}$, where $h_i : \mathcal{K} \to [n]$ and the $\mathsf{hint}_\mathsf{kvs}$.

- $\mathsf{KVS.Decode}(\mathsf{pp}, k, \mathsf{hint}_\mathsf{kvs}, \mathsf{DB}[\mathcal{H}(k)]) \to v$: Given the parameters pp, a key $k \in \mathcal{K}$, the $\mathsf{hint}_\mathsf{kvs}$, and the element locate at $\{h_i(k)\}_{i \in [\alpha]}$ in DB, output a value $v \in \{\mathcal{V} \cup \perp\}$.

The correctness of KVS is defined as follows.

- **Inclusion Correctness:** If the $(k, v) \in \mathsf{KV}$, then $\mathsf{KVS.Decode}(\mathsf{pp}, \mathcal{H}, k, \mathsf{hint}_\mathsf{kvs}, \mathsf{DB}[\mathcal{H}(k)]) = v$ with the probability 1.

- **Exclusion Correctness:** For any $(k, \cdot) \notin \mathsf{KV}$, the $\mathsf{KVS.Decode}(\mathsf{pp}, \mathcal{H}, k, \mathsf{hint}_\mathsf{kvs}, \mathsf{DB}[\mathcal{H}(k)]) = \perp$ with the probability $1 - \varepsilon$.

Instantiation BFF-KVS. The details of BFF [19] are presented in Figure 6 in the Appendix. The original scheme employs a fingerprinting function defined as $\mathsf{fpt}(k, v) = \mathsf{hash}(k)$, and determines whether the output is 1 or $\perp$ by checking

$$\mathsf{hash}(k') \stackrel{?}{=} \mathsf{DB}[h_0(k)] \oplus \cdots \oplus \mathsf{DB}[h_{\gamma-1}(k)] \quad (5)$$

where $\mathsf{hash} \colon \mathcal{K} \to \{0,1\}^{\mu_0}$. False positives may occur due to collisions in hash, which can be mitigated by appropriately setting the output length of hash. ChalametPIR [9] and KPIR [34] employ a fingerprint function defined as $\mathsf{fpt}(k, v) = \mathsf{hash}(k) \| v$. then checking

$$\mathsf{hash}(k') \| v \stackrel{?}{=} \mathsf{DB}[h_0(k)] \oplus \cdots \oplus \mathsf{DB}[h_{\gamma-1}(k)] \quad (6)$$

In this construction, the pairs in KV are encoded into DB which guarantees inclusion correctness. The exclusion correctness are holds with probability $1 - 2^{-\mu_0}$.

2.3 Minimal Perfect Hash Function

The Perfect Hash Function (PHF) [16, 25, 27, 29, 41] is given a set $S = \{k_0, ..., k_{m-1}\} \subseteq \mathcal{K}^m$ within a universe $\mathcal{K}$, a perfect hash function $\mathsf{PHF} : S \to [m']$, maps all keys in S to $[m'] = \{0, \ldots, m'-1\}$ without collisions. For $k \notin S$, the function may return arbitrary values in $[m']$. A Minimal Perfect Hash Function (MPHF) corresponds to $m' = m$, , using the following three algorithms:

- $\mathsf{MPHF.Setup}(\omega) \to \mathsf{pp}$: Given the radio ω bits per key, output the parameters pp.

- $\mathsf{MPHF.Encode}(\mathsf{pp}, S) \to (\mathsf{hint}_\mathsf{mphf}, S')$: Given the parameters pp, the key sets S, output the MPHF $\mathsf{hint}_\mathsf{mphf}$ and an rearranged set S'.

- $\mathsf{MPHF.Decode}(\mathsf{pp}, \mathsf{hint}_\mathsf{mphf}, k) \to i$: Given the parameters pp, the $\mathsf{hint}_\mathsf{mphf}$, a key $k \in S'$, output a index $i \in [m]$.

The correctness of MPHF is defined as follows.

- **Inclusion Correctness:** If the key $k = k_i \in S'$, then $\mathsf{MPHF.Decode}(\mathsf{pp}, \mathsf{hint}_\mathsf{mphf}, k) = i$ with probability 1.

The sets S and S' are identical as sets, but differ in the order of their elements. The space lower bounds of MPHF is about $\omega_{\mathsf{min}} \approx 1.443$ bits per key, the Consensus RecSplit [27] can achieve around 1.444 bits per key, while BBHash [29] requires around 2.718 bits per key but offers a fast encoding time.

3 Key-Value Store from MPHF

In this section, we show have to construct the MPHF-KVS, we overview the MPHF firstly, and choose two typical method: BBHash [29] and Consensus RecSplit [27] to instantiate the MPHE-KVS, the first have a short encoding time and large memory usage, and the later have a slow encoding time and nearly optimal memory usage.

3.1 Overview of MPHF

The core problem of Minimal Perfect Hash Function (MPHF) [25] is the following: given a set $S = \{k_0, \ldots, k_{m-1}\}$, construct a bijective function $\mathsf{MPHF} : S \to [m]$. For any key $k \notin S$, the function may return an arbitrary index in $[m]$. Thus, every key in S is mapped to a unique index in a compact index set, while an MPHF by itself does not support membership queries and only handles static databases.

There are three main approaches to constructing MPHF. The first approach [26] is based on explicit key-to-index mappings, where the position of each key is encoded using $O(\log_2 m)$ bits per key. The second approach [27,41] relies on brute-force search. Since there are m^m possible functions mapping S to $[m]$ and only $m!$ of them are bijective, the success probability of randomly selecting a bijection is $\frac{m!}{m^m}$. Using Stirling's approximation,

$$m! \approx \sqrt{2\pi m}\left(\frac{m}{e}\right)^m$$

this probability can be approximated as

$$\frac{m!}{m^m} \approx \frac{\sqrt{2\pi m}}{e^m}$$

which is negligible for sufficiently large m. The third approach [29] exploits hash collisions by assigning each key a short fingerprint and recursively resolving collisions. Due to the birthday paradox, the recursion can terminate easily.

While these three methods [26,27,29,41] have led to significant improvements over the original approaches for constructing MPHF, they differ substantially in terms of space efficiency, construction time, and practical applicability.

The evaluation metrics include memory usage, construction time, query time, dataset size, and the trade-offs between these aspects. An interesting observation concerns the space lower bound for representing an MPHF [25]. Let $\mathcal{K}$ be the set of keys with $|\mathcal{K}| = u$, and let f be an MPHF defined on the domain $\mathcal{K}$ with range $[m]$. Consider a set S obtained by choosing one element from each preimage $f^{-1}(0), \ldots, f^{-1}(m-1)$. The largest S numbers is $(u/m)^m$ when all preimages have equal size, i.e., $|f^{-1}(i)| = u/m$. The total number of such sets is $\binom{u}{m}$, therefore, the minimal number of distinguishable functions is $\binom{u}{m}/(u/m)^m$. Taking logarithms gives the information-theoretic lower bound per key:

$$\log_2 \frac{\binom{u}{m}}{(u/m)^m} \approx n\log_2 e - O(\log n) \approx 1.443 \text{ bits per key.}$$

3.2 A MPHF example

We explain the core idea of the BBHash and Consensus RecSplit below, for more details, please refer to the original works [27,29].

BBHash. It constructs an MPHF by exploiting hash collisions. Suppose we have the set $S = \{k_0, \ldots, k_4\}$ and a hash function $h_0 : S \to [5]$ such that $h_0([k_0, \ldots, k_4]) = [1,3,4,3,2]$.

In A_0, we define an array A_0 where $A_0[h_0(k_i)] = 1$ if there is no collision at that position. Thus, we have $A_0 = [0,1,1,0,1]$ due to the collision $h_0(k_1) = h_0(k_3) = 3$.

In A_1, we choose a second hash function $h_1 : S \to [2]$ for the colliding keys $\{k_1, k_3\}$ such that $h_1([k_1, k_3]) = [0,1]$, resulting in $A_1 = [1,1]$ with no collisions.

The BBHash $\mathsf{hint}_{\mathsf{mphf}} = A_0 \| A_1$, and the index of a key k is determined by the position of the first 1 in $A_0[h_0(k)], A_1[h_1(k)]$, we rearrange the $S' = \{k_0, k_4, k_2, k_1, k_3\}$.

3.3 Instantiation MPHF-KVS

An MPHE aims to find a bijective function $\mathsf{MPHE} : S \to [m]$ to locate the positions of keys. The original BFF algorithm [19] only supports membership queries and can be unified with KVS by defining $\mathsf{fpt}(k, v) = \mathsf{hash}(k) \| v$. In our MPHE, we support

$$\mathsf{MPHF}(k'_i) = i$$

where $k'_i \in S'$. Each subroutine of KVS invoke the subroutine of MPHF, in the Encode step, we extract the value set DB from the KV in the order induced by the key set S', ensuring that $\mathsf{DB}[i] = v'_i$. However, our construction does not inherently support membership queries. To address this issue, we store $\mathsf{DB}[i] = \mathsf{fpt}_\varepsilon(k'_i, v'_i)$, where $\mathsf{fpt}_\varepsilon(k, v) = \mathsf{hash}_1(k) \| v$, $\mathsf{hash}_1 \colon \mathcal{K} \to \{0,1\}^{\mu_1}$. where the collision resistance of $\mathsf{hash}_1(\cdot)$ enables membership verification. Nevertheless, even if a queried key k' satisfies $\mathsf{hash}_1(k') \neq \mathsf{hash}_1(k'_i)$, the value v'_i would still be revealed. To prevent this leakage, we change the fingerprint function to

$$\mathsf{fpt}_\varepsilon(k, v) = \mathsf{hash}_1(k) \| (\mathsf{hash}_2(k) \oplus v) \tag{7}$$

where $\mathsf{hash}_2 \colon \mathcal{K} \to \{0,1\}^{\log|\mathcal{V}|}$.

In the Decode step, it determines whether the output is v or $\perp$ by checking

$$\mathsf{fpt}_\varepsilon(k', v) \stackrel{?}{=} \mathsf{DB}[\mathsf{MPHF}(k)] \tag{8}$$

The inclusion correctness of MPHF-KVS is guarantees by the inclusion correctness of MPHF, the exclusion correctness are holds with probability $1 - 2^{-\eta_1}$.

4 Batch PIR

In this section, we first define Batch PIR and describe the basic structure of SI-PIR. We then show how to transform these

schemes into Batch PIR using the Rewind & Skip techniques, and present three concrete instantiations from Piano [46], SinglePass [4], and WR25 [44].

4.1 Definition of Batch PIR

A Batch PIR (BPIR) allows multiple elements to be queried simultaneously. Given as input a database $\mathsf{DB} = \{\alpha_i\}_{i\in[n]} \in \mathcal{Z}^n$, it is defined by the following algorithms:

$\mathsf{BPIR.Setup}(1^\lambda, 1^\kappa, \mathsf{DB}) \to (\mathsf{hint_s}, \mathsf{hint_c})$: Given the database DB, outputs the server-side $\mathsf{hint_s}$, client-side $\mathsf{hint_c}$.

$\mathsf{BPIR.Query}(\mathsf{hint}_c, x) \to (\mathsf{st}, q)$: Given the query indices $x = \{x_0, x_1, \ldots, x_{k-1}\} \in [n]^k$, outputs a client state st and batch queries q.

$\mathsf{BPIR.Answer}(\mathsf{hint_s}, \mathsf{DB}, q) \to a$: Upon receiving the batch queries q, with the server-side $\mathsf{hint_s}$ and database DB, outputs the batch answers a.

$\mathsf{BPIR.Reconstruct}(\mathsf{hint_c}, \mathsf{st}, a) \to \beta$: Upon receiving the batch answers a, with the client's $\mathsf{hint_c}$ and state st, output the elements $\beta \in \mathcal{Z}^k$. The hint and state are then updated as follows: $(\mathsf{hint_s}, \mathsf{hint_c}) \leftarrow \mathsf{BPIR.Refresh}(\mathsf{hint_s}, \mathsf{hint_c}, \mathsf{st}, a)$ where $\mathsf{BPIR.Refresh}$ is a deterministic function.

Correctness and Security of BPIR are as follows.

Definition 3.1. (Correctness) A BPIR scheme is correct if, for any database $\mathsf{DB} \in \mathcal{Z}^n$ and any query indices $x \in [n]^k$ the following holds:

$\mathsf{BPIR.Reconstruct}(\mathsf{hint_c}, \mathsf{st}, \mathsf{BPIR.Answer}(q, \mathsf{hint}_s)) = \mathsf{DB}[x]$

where $(\mathsf{hint_s}, \mathsf{hint}_c) \leftarrow \mathsf{BPIR.Setup}(1^\lambda, 1^\kappa, \mathsf{DB})$ and $(\mathsf{st}, q) \leftarrow \mathsf{BPIR.Query}(\mathsf{hint}_c, x)$ with probability at least $1 - \mathsf{negl}(\lambda) - \mathsf{negl}(\kappa)$. If the query indices $x \notin [n]^k$, then the reconstruct algorithm outputs $\perp$ with probability 1.

Definition 3.2. (Correctness up to L queries) A BPIR is said to be correctness up to L queries, if it guarantees correctness for at most L queries, and outputs $\perp$ with probability 1 once the queries exceeds L.

Definition 3.3. (Security) A BPIR scheme is secure if, for any polynomial time adversary $\mathcal{A}$ and for any indices $(x^0, x^1) \in [n]^k$, the following holds:

$$\begin{aligned}&\big|\Pr[\mathcal{A}(q) = 1 \mid (\mathsf{st}, q) \leftarrow \mathsf{BPIR.Query}(\mathsf{hint}_c, x^0)] \\ &-\Pr[\mathcal{A}(q) = 1 \mid (\mathsf{st}, q) \leftarrow \mathsf{BPIR.Query}(\mathsf{hint}_c, x^1)]\big| \le \mathsf{negl}(\kappa)\end{aligned} \tag{9}$$

Without loss of generality, by setting $k = 1$, the BPIR degenerates to the Index PIR. Unlike traditional PIR schemes that retrieve only a single record at a time, BPIR enables querying multiple random positions in the database simultaneously.

4.2 The Basic Structure of SI-PIR

We review the general structure of SI-PIR, which is followed by many works [4, 5, 12, 44, 46].

A Toy Scheme. Consider a server holding a database $\mathsf{DB} = \{\alpha_i\}_{i\in[n]} \in \mathcal{Z}^n$, and a client that wishes to retrieve a specific element without revealing the queried index to the server.

This scheme proceeds in two phases:

Offline phase. The client can freely read any value from the database DB.

- The client randomly selects multiple subsets $(S_0, S_1, \ldots, S_{M_1})$ from $\{0, 1, \ldots, n-1\}$, each of size $\sqrt{n}$, computes $h_j \leftarrow \oplus_{i\in S_j}\alpha_i$, and stores the pairs $(S_j, h_j)_{j\in[M_1]}$ as client's hint.
- The client delete the database DB.

Online phase. The client queries the index $i \in [n]$.

- The client identifies a subset S_j that contains the desired index i and sends $S' \leftarrow S_j \setminus \{i\}$ to the server.
- The server computes $h \leftarrow \oplus_{i\in S'}\alpha_i$, sends it to the client.
- The client recover the $\alpha_i \leftarrow h_j \oplus h$.
- The client either deletes the pair (S_j, h_j) or refreshes it to a new pair in the client's hint.

We observe that the server-side online computation requires only $\sqrt{n}$ XOR operations, resulting in sublinear computational cost, while the online communication cost is also $\sqrt{n}$. This efficiency stems from the offline phase, which incurs linear computation and communication costs but is performed locally on the client. Since the offline phase serves as an initialize phase, we assume it is executed only once.

Without loss of generality, we provide a heuristic view to fix the bugs in toy schemes step by step. First, we need to ensure that the client's query index is contained in at least one subset S_j. This can be achieved either by constructing disjoint sets $(S_0, S_1, \ldots, S_{\sqrt{n}-1})$, each of size $\sqrt{n}$ [4], or by generating additional random subsets $(S_0, S_1, \ldots)$ to cover the entire index set $[n]$ [46]. Second, we need to ensure that the client's query remains unbiased. Since the set S' sent by the client always excludes the queried index i, the server can infer that i is not contained in S'. This issue can be mitigated by introducing a small error probability [12]: the client sends the actual set S' with probability $1 - 1/\sqrt{n}$, and otherwise sends a random set to obscure the true query.

Third, we need to address how the client can replenish consumed sets S_j to support multiple queries. Once a set S_j is used to construct S', it should not be reused, as the server could compare different instances to infer the queried index. To mitigate this, the client can either prestore multiple pairs (S_j, h_j) to replace the consumed ones or refresh them by querying another server. Fourth, we need to address how to reduce the client's storage overhead. The hint $(S_j, h_j)_{j\in[M_1]}$ must at minimum cover the entire database, which implies that its total size may exceed that of the original database. However, since

BPIR.Setup$(1^\lambda, 1^\kappa, \mathsf{DB}) \to \mathsf{hint}$.

- Return $\mathsf{hint} \leftarrow \mathsf{Setup}(1^\lambda, 1^\kappa, \mathsf{DB})$.

BPIR.Query$(\mathsf{hint}, x) \to (\mathsf{st}, q)$.

- Store the hint.
- Parse $(x_0, x_1, \ldots, x_{k-1}) \leftarrow x$, for each x_i:
  - Let $(\mathsf{st}_i, q_i) \leftarrow \mathsf{Query}(\mathsf{hint}, x_i)$
  - Update $\mathsf{hint} \leftarrow \mathsf{FakeRefresh}(\mathsf{hint}, \mathsf{st}_i)$
- Let $\mathsf{st} = (\mathsf{st}_0, \mathsf{st}_1, \ldots, \mathsf{st}_{k-1})$.
- Return $(\mathsf{st}, q = (q_0, q_1, \ldots, q_{k-1}))$.

BPIR.Answer$(\mathsf{DB}, q) \to a$.

- Parse $(q_0, q_1, \ldots, q_{k-1}) \leftarrow q$, for each q_i:
  - let $a_i \leftarrow \mathsf{Answer}(\mathsf{DB}, q_i)$.
- Return $a = (a_0, a_1, \ldots, a_{k-1})$.

BPIR.Reconstruct$(\mathsf{hint}, \mathsf{st}, a) \to \beta$.

- Rewind to the previously stored hint.
- Parse $(a_0, a_1, \ldots, a_{k-1}) \leftarrow a$, for each a_i:
  - Let $\beta_i \leftarrow \mathsf{Reconstruct}(\mathsf{hint}, \mathsf{st}_i, a_i)$.
- Return $\beta = (\beta_0, \beta_1, \ldots, \beta_{k-1})$.

Figure 2: Batch SI-PIR

each S_j is a randomly generated set, it can be represented using a random seed to reduce storage requirements.

Next, we consider the construction of BPIR within this type PIR, focusing specifically on retrieving random elements in a single round. In contrast to SealPIR [3], which relies on Cuckoo filters and requires querying each bucket to save the overall computational cost. Moreover, unlike SimplePIR [21], KOPIR [24], and Row-KOPIR [34], where batched entries are inherently correlated, our batched indices are generated independently and remain uncorrelated.

The challenge lies in the fact that the pairs (S_j, h_j) may be deleted or refreshed, causing the set of $(S_j, h_j)_{j \in [M_1]}$ to change across different queries. However, the key insight is that a query only consists of an index set, and its generation is independent of the server's response (i.e., the partial sum). It only depends on the client's hint, a nonce, and the former query. Therefore, batch queries can be generated simultaneously.

4.3 Rewind & Skip technique

In modern computing, virtualization is a well-established technology. In virtual machines, the rewind technique is realized through snapshots [40], which enable users to save the state of a virtual machine at a specific point in time and revert to it when needed. Similarly, the rewind technique is employed by simulators to construct computationally indistinguishable views in simulation-based proofs [30]. This technique can be illustrated using a Turing machine, which consists of a storage tape and a controller; the controller can save its current state and later rewind to this saved state at any point in the future.

We propose that the rewind technique serves a same purpose in a program: it is used to restore the state of a subprocess to a previous point in time. Meanwhile, we propose that the Skip technique involves the controller making multiple copies of its current state. When one copy is used, it switches to another. After the task is completed, all used copies are refreshed to maintain consistency.

4.4 Instantiation Batch SI-PIR

In Figure 2, we present a general framework for BPIR by applying the Rewind & Skip techniques. We then illustrate the FakeRefresh procedure and provide details for three representative schemes: Piano [46], SinglePass [4], and WR25 [44], to complete our BPIR construction. The blue text indicates the Rewind operations, while the red text indicates Skip operations (specific to Piano). Either of these techniques can be employed to extend the scheme into a Batch SI-PIR.

Piano [46] is a single-server PIR scheme that achieves sublinear server-side computation. It maintains primary tables, backup tables, and replacement entries as the client's hint. During a query, the client first locates the target index in the primary tables and uses it to construct the query set. In the reconstruct phase, the backup tables and replacement entries are then used to replenish the consumed entry in the primary tables. In Figure 3, we illustrate the original Piano extended with the Rewind & Skip techniques, which serves as a subroutine in constructing the Batch SI-PIR.

In the Rewind process, during the query phase, the client first stores the current hint. It then updates the primary table without relying on the server's response a, and records the input nonce and other intermediate states. Upon receiving the server's response, the client reconstructs each β_i by reusing the previously stored hint, and the same nonce in the states, thereby ensuring that the update steps in both the query and reconstruct phases remain consistent. In the Skip process, the client initially marks all entries in the primary table as unused during the setup phase. For each batch query, it then guarantees that no two queries access the same $\mathbb{T}_i$, thereby preventing collisions within the batch queries.

SingePass [4] is a two-server PIR scheme that relies on the assumption of non-colluding servers. After each query,

the client updates each permutation P_j and partial sums $h = (h_0, \ldots, h_{m-1})$ to prepare for the next query. Since the P_j are disjoint and each index appears once, only the Rewind technique can be applied to implement the batch version. The subroutine is illustrated in Figure 4. By employing the Rewind technique, the client change the permutation P_j after each query and records the client state of st. Upon receiving the response a, the client synchronizes updates with the client state st. Since the reconstruct step does not involve the permutation P_j, there is no need to rewind the P_j, instead, only the internal client state st changes need to be replayed.

Singlepass supports an unbounded number of queries, and therefore its batch version can also handle an unlimited number of queries. However, it has certain limitations. During execution, the client's hint grows roughly to the same scale as the database.Moreover, if the program terminates and later reruns, reconstructing the updated permutation P_j from a random seed sk and nonce is difficult, resulting in significant storage overhead on the client side.

WR25 [44] proposed a PIR scheme that improves upon SinglePass [4] by introducing dummy placeholders and a relocation data structure. This allows operation in a single-server setting, albeit with a bounded number of queries. Additionally, generating nonces from a seed keeps the client's hint small even if the program terminates. Figure 4 illustrates the subroutine of the batch version. Similar to SinglePass, we can use the Rewind technique to record the globally shared Hist and the client state st. During the reconstruct phase, Hist is modified by invoking $\mathsf{Hist.Append}(c)$. To maintain synchronization, FakeRefresh also invoke $\mathsf{Hist.Append}(c)$. Then upon receiving the response, we rewind Hist, replay st, and keep Hist updated in a synchronized manner.

4.5 Security Analysis

There are four Batch PIR schemes: Piano with Skip, Piano with Rewind, SinglePass with Rewind, and WR25 with Rewind.

Since the Rewind technique does not alter the input or output of the subroutine, the correctness of the latter three schemes directly follows from that of their original versions. The only slight difference arises in Piano with Skip, where multiple distinct hits on the primary table reduce the success probability, as analyzed in Theorem 3.1. Moreover, Piano and WR25 support only a limited number of queries: their correctness holds for up to L queries, but beyond L queries the scheme fails due to insufficient material, and outputs $\perp$ with probability 1. For security, we need to show that these schemes satisfy Definition 3.3. Since Piano already satisfies the security requirements of its single-query version, its batch version follows directly from Theorem 3.2. The same argument applies to SinglePass and WR25.

Theorem 3.1. (Correctness) Assume n is bounded by $\mathsf{poly}(\lambda)$ and $\mathsf{poly}(\kappa)$. The $\alpha(\kappa)$ be any superconstant function. Setting $M_1 = k - 1 + \sqrt{n}\ln\kappa\alpha(\kappa)$, $M_2 = 3 \cdot \ln\kappa\alpha(\kappa)$, all $Q = 1/k \cdot \sqrt{n}\ln\kappa\alpha(\kappa)$ queries in the Piano with Skip technique shown in Figure 3 are answered correctly with probability at least $1 - \mathsf{negl}(\lambda) - \mathsf{negl}(\kappa)$.

Proof. First, we need to prove that the probability of the client successfully retrieving all k elements in a batch query is at least $1 - \mathsf{negl}(\kappa)$. We assume that the sets $\{S_i\}_{i\in[M_1]}$ are identically distributed, where each set contains the queried element with probability $1/\sqrt{n}$.

Event_1 : a specific element does not appear in M_1 sets.

$\Pr[\mathsf{Event}_1] = \left(1 - \frac{1}{\sqrt{n}}\right)^{M_1}$

Event_2 : a random k element at least one does not appear in M_1 sets.

$\Pr[\mathsf{Event}_2] = 1 - (1 - \Pr[\mathsf{Event}_1])^k$

Event_3 : a random k element at least one does not appear in M_1 sets, in sampling without replacement.

$$\Pr[\mathsf{Event}_3] = 1 - \prod_{i=0}^{k-1}\left(1 - \left(1 - \frac{1}{\sqrt{n}}\right)^{M_1 - i}\right)$$
$$< 1 - \left(1 - \left(1 - \frac{1}{\sqrt{n}}\right)^{M_1 - k + 1}\right)^k$$
$$< k \cdot \left(1 - \frac{1}{\sqrt{n}}\right)^{M_1 - k + 1}$$
$$\leq k \cdot \frac{1}{\kappa^{\alpha(\kappa)}}$$

Event_4 : a random Q batch queries at least one failure in M_1 sets.

$\Pr[\mathsf{Event}_4] = 1 - (1 - \Pr[\mathsf{Event}_3])^Q < Q \cdot \Pr[\mathsf{Event}_3]$

So, the failure probability in one batch query not exceed k times, so it is negligible when k is small. And, the final failure probability after Q times queries is at most $\frac{\sqrt{n}\ln\kappa\alpha(\kappa)}{\kappa^{\alpha(\kappa)}}$, it is a $\mathsf{negl}(\kappa)$ since n is bounded by $\mathsf{poly}(\kappa)$.

Second, the client uses a random oracle to sample the sets instead of a PRF, the failure probability is $\mathsf{negl}(\lambda)$. Therefore, all batch queries are correctly with probability at least $1 - \mathsf{negl}(\lambda) - \mathsf{negl}(\kappa)$. □

Theorem 3.2. (Security) If the Piano [46] is secure for a single query, then the BPIR scheme shown in Figure 2 with the subroutine in Figure 3, satisfies Definition 3.3.

Proof. The Piano is secure for a single query means that it satisfies the following property: for any probabilistic polynomial-time adversary $\mathcal{A}$ acting as the server, and for any polynomially bounded parameters n and Q, the views of $\mathcal{A}$ during the real execution of the protocol are computationally indistinguishable from those in an ideal world interaction, where $\mathsf{Sim}(1^\lambda, n)$ interacts with $\mathcal{A}$ without accessing $x \in [n]$. Therefore, it is clear that any probabilistic polynomial-time adversary $\mathcal{A}'$ cannot distinguish between any pair of inputs $(x^0, x^1) \in [n]^k$. If not, there would exist an adversary $\mathcal{A}'$ capable of distinguish at least one pair $(x_i^0, x_i^1) \in [n]$. In this case, we could construct an adversary $\mathcal{A}$ that invokes $\mathcal{A}'$ to distinguish between the real and ideal worlds. Hence, the BPIR scheme with the subroutines shown is Figure 3 is secure. □

The correctness of the BPIR illustrated in Figure 4 follows directly, and their security is guaranteed by the security of the

Construction 1 Single-Server Scheme for $Q = \sqrt{n}\log\kappa\cdot\alpha(\kappa)$ Queries. Parameterized by database size n, we divide DB into $T = \sqrt{n}$ rows of size $m = \sqrt{n}$ while $T|n$, and define $\mathsf{DB}_j = \mathsf{DB}[j\cdot m : (j+1)\cdot m]$ to be the j-th row of the database. The client store the hint include a key sk, and $h = (h_0, h_1, \ldots, h_{m-1})$. We implicitly pass client state status $\mathsf{st} = (i^*, ind, r)$ from Query to Reconstruct and implicitly parse $\mathsf{hint} = (\mathsf{sk}, h)$. We use $\alpha(\kappa)$ to denote an arbitrarily small super-constant function. For $i \in [n]$, let $\mathsf{chunk}(i) = \lfloor i/m \rfloor$ be the chunk i belongs to. A pseudorandom function $\mathsf{PRF}(\mathsf{sk}, j) \in [\sqrt{n}]$, $\mathsf{Set}(\mathsf{sk}) := \{j\cdot\sqrt{n} + \mathsf{PRF}_{\mathsf{sk}}(j)\}_{j\in[\sqrt{n}]}$. $\mathsf{Set}(\mathsf{sk}, x)$ where $x \in \{\perp\}\cup[n]$ is defined as follows: $\mathsf{Set}(\mathsf{sk}, x) = \mathsf{Set}(\mathsf{sk})$ if $x = \perp$, $\mathsf{Set}(\mathsf{sk}, x) = \mathsf{Set}(\mathsf{sk})\langle\mathsf{chunk}(x) \to x\rangle$ otherwise.

Setup$(1^\lambda, 1^\kappa, \mathsf{DB}) \to \mathsf{hint}$**.**

- For each $i \in [M_1]$, initialize $\mathsf{sk}_i \leftarrow^{\$} \{0,1\}^\lambda$, $h_i \leftarrow 0$, where $M_1 = \sqrt{n}\log\kappa\cdot\alpha(\kappa)$.
- For each $j \in [T]$, $i \in [M_2]$, initialize $\overline{\mathsf{sk}}_{j,i} \leftarrow^{\$} \{0,1\}^\lambda$, $\overline{h}_{j,i} \leftarrow 0$, where $M_2 = \log\kappa\cdot\alpha(\kappa)$.
- The client streams the database by each element. For j-th chunk DB_j:
  - For each $i \in [M_1]$, update $h_i \leftarrow h_i \oplus \mathsf{DB}[\mathsf{Set}(\mathsf{sk}_i)[j]]$.
  - For each $i \in [M_2]$, store $(r, \mathsf{DB}[r])$, where $r \leftarrow^{\$} j\cdot T + [T]$ is a random index from the j-th chunk.
  - For each $i \in [T]\setminus\{j\}$ and $c \in [M_2]$, let $\overline{h}_{i,c} \leftarrow \overline{h}_{i,c} \oplus \mathsf{DB}[\mathsf{Set}(\overline{\mathsf{sk}}_{i,c})[j]]$.
  - Delete $\mathsf{DB}[j\cdot m : (j+1)\cdot m]$ from the local storage.
- Let $\mathbb{T} = \{((\mathsf{sk}_i, \perp), h_i)\}_{i\in[M_1]}$ denote the client's primary table, sign all $\mathbb{T}_i$ are unused, let $\mathbb{B} = \{(\overline{\mathsf{sk}}_{j,i}, \overline{h}_{j,i})\}_{i\in[M_2]}$ denote the backup table for the j-th chunk, and let $\mathbb{R} = (r, \mathsf{DB}[r])_{i\in[M_2]}$ denote the replacement entries.
- Return $\mathsf{hint} = (\mathbb{T}, \mathbb{B}, \mathbb{R})$.

Query$(\mathsf{hint}, x) \to (\mathsf{st}, q)$**.**

- Finds a unused hint $T_i := ((\mathsf{sk}_i, x'), h_i)$ in its primary table T such that $x_i \in \mathsf{Set}(\mathsf{sk}_i, x')$. Let $S = \mathsf{Set}(\mathsf{sk}_i, x')$. Sign T_i is used.
- Let $j^* = \mathsf{chunk}(x)$, Finds the first unconsumed replacement entry from the j^*-th chunk, denoted $(r, \mathsf{DB}[r])$.
- Return $(\mathsf{st} = (h, j^*, r, q), q)$.

Answer$(\mathsf{DB}, q) \to a$**.**

- Return $a = \bigoplus_{j\in q} \mathsf{DB}[j]$.

Reconstruct$(\mathsf{hint}, \mathsf{st}, a) \to \beta$**.**

- Let $\beta = q \oplus h_i \oplus \mathsf{DB}[r]$.
- Finds the next unconsumed backup entry $(\overline{\mathsf{sk}}_{j^*,c}, \overline{h}_{j^*,c})$ belonging to the j^*-th chunk. If not found, Generates a random $\overline{\mathsf{sk}}_{j^*,c}$ and lets $\overline{h}_{j^*,c} = 0$.
- Replaces the matched entry in the primary table with $((\overline{\mathsf{sk}}_{j^*,c}, x), \overline{h}_{j^*,c} \oplus \beta)$. Sign it unused.
- Return β.

FakeRefresh$(\mathsf{hint}, \mathsf{st}) \to \mathsf{hint}$.

- Parse $(h, j^*, r, q) \leftarrow \mathsf{st}$. Finds the next unconsumed backup entry $(\overline{\mathsf{sk}}_{j^*,c}, \overline{h}_{j^*,c})$ belonging to the j^*-th chunk. If not found, Generates a random $\overline{\mathsf{sk}}_{j^*,c}$ and lets $\overline{h}_{j^*,c} = 0$.
- Replaces the matched entry in the primary table with $((\overline{\mathsf{sk}}_{j^*,c}, x), \overline{h}_{j^*,c})$.

Figure 3: Piano [46] with Rewind & Skip techniques

Construction 2 Let n be the database size. We divide DB into T rows of length $m = n/T$ (assuming $T \mid n$), and define the j-th row as $\mathsf{DB}_j = \mathsf{DB}[j \cdot m : (j+1) \cdot m]$. The client stores a $\mathsf{hint} = (\mathsf{sk}, h)$, where sk is a key and partial sum h with size m. For each P_j, we instantiate the $\mathsf{PRP}(\mathsf{sk}_j, \cdot)$ using $\mathsf{sk}_j = \mathsf{PRF}(\mathsf{sk}, j)$.

Setup$(1^\lambda, \mathsf{DB}) \to \mathsf{hint}$**.**

- Initialize a random key: $\mathsf{sk} \leftarrow^{\$} \{0,1\}^\lambda$ and $h = (h_0, \ldots, h_{m-1}) \in 0^m$.
- For each $j \in [T]$, let $P_j \leftarrow \mathsf{PRP}(\mathsf{sk}_j, \cdot)$ over $[m]$, where $\mathsf{sk}_j = \mathsf{PRF}(\mathsf{sk}, j)$.
- For each $P_j(i)$-th element in row j, update $h_i \leftarrow h_i \oplus \mathsf{DB}_j[P_j(i)]$.
- Return $\mathsf{hint} = (\mathsf{sk}, h)$.

Query$(\mathsf{hint}, x) \to (\mathsf{st}, q)$**.**

- Let $(i^*, j^*) \leftarrow (x \bmod m, \lfloor x/m \rfloor)$.
- Find $\mathsf{ind} \in [m]$ such that $P_{i^*}(\mathsf{ind})$ is equal to j^*.
- Set $q^1 = [P_i(\mathsf{ind}) : i \in [T]]$, $q^1[i^*] \leftarrow r^* \in [m]$.
- Set $q^0 = [P_i(r_i) : i \in [T]]$ for random $r_i \in [m]$.
- For each $i \in [T], i \neq i^*$, swap $P_i(\mathsf{ind})$ and $P_i(r_i)$.
- Return $(\mathsf{st} = (i^*, \mathsf{ind}, (r_0, \ldots, r_{T-1})), (q^0, q^1))$.

Answer$(\mathsf{DB}, q) \to a$**.**

- For $q = (q^0, q^1)$:
  - $a^0 = [\mathsf{DB}_0[q^0[0]], \ldots, \mathsf{DB}_{k-1}[q^0[T-1]]]$.
  - $a^1 = [\mathsf{DB}_0[q^1[0]], \ldots, \mathsf{DB}_{k-1}[q^1[T-1]]]$.
- Return $a = (a_0, a_1)$.

Reconstruct$(\mathsf{hint}, \mathsf{st}, a) \to \beta$**.**

- Compute $\beta = \left(\bigoplus_{i \in [T], i \neq i^*} a^1[i]\right) \oplus h_{\mathsf{ind}}$.
- For each $i \in [T], i \neq i^*$, update:

$$h_{\mathsf{ind}} \leftarrow h_{\mathsf{ind}} \oplus a^0[i] \oplus a^1[i],$$
$$h_{r_i} \leftarrow h_{r_i} \oplus a^0[i] \oplus a^1[i].$$

- Return β.

FakeRefresh$(\mathsf{hint}, \mathsf{st}) \to \mathsf{hint}$.

- Do nothing.

(a) SinglePass

Construction 3 Let n be the database size. We divide DB into T rows of length $m = n/T$ (assuming $T \mid n$), and define the j-th row as $\mathsf{DB}_j = \mathsf{DB}[j \cdot m : (j+1) \cdot m]$. The client stores $\mathsf{hint} = (\mathsf{sk}, \mathsf{Hist}, h)$, where sk is a key, Hist tracks the history of consumed columns, and partial sum h with size $m' = 2m$. The DS from Constructions 3.1 and 3.2 in appendix is instantiated with $\mathsf{PRP}(\mathsf{sk}_j, \cdot)$ using $\mathsf{sk}_j = \mathsf{PRF}(\mathsf{sk}, j)$.

Setup$(1^\lambda, \mathsf{DB}) \to \mathsf{hint}$**.**

- Initialize a random key: $\mathsf{sk} \leftarrow^{\$} \{0,1\}^\lambda$ and $h = (h_0, \ldots, h_{m'-1}) \in 0^{m'}$.
- Call $\mathsf{DS.Init}(\mathsf{sk}, T, m')$, and set $\mathsf{Hist} = \mathsf{DS.Hist}$.
- For each element $\mathsf{DB}_j[e]$ in row j, update $c \leftarrow \mathsf{DS.Locate}(j, e)$ and compute $h_c \leftarrow h_c \oplus \mathsf{DB}_j[e]$.
- Return $\mathsf{hint} = (\mathsf{sk}, \mathsf{Hist}, h)$.

Query$(\mathsf{hint}, x) \to (\mathsf{st}, q)$.

- Let $c = \mathsf{DS.Locate}(j^*, x \bmod m)$, $j^* = \lfloor x/m \rfloor$. $q = (\mathsf{DS.Access}(0, c), \ldots, \mathsf{DS.Access}(T-1, c))$.
- Choose a random $r \in [m'] \setminus C$, and replace $q[j^*] \leftarrow \mathsf{DS.Access}(j^*, r)$.
- Return $(\mathsf{st} = (x, j^*, c, q, r), q)$.

Answer$(\mathsf{DB}, q) \to a$**.**

- Return $a = (\mathsf{DB}_0[q[0]], \ldots, \mathsf{DB}_{T-1}[q[T-1]])$.

Reconstruct$(\mathsf{hint}, \mathsf{st}, a) \to \beta$.

- Compute $\beta = h[c] \oplus \bigoplus_{j \in [T], j \neq j^*} a[j]$.
- Replace $a[j^*] \leftarrow \beta$ and set $q[j^*] \leftarrow x \bmod m$.
- Call $\mathsf{Hist.Append}(c)$.
- For each $j \in [T]$ where $q[j] \neq \perp$:
  - Let $c = \mathsf{DS.Locate}(j, q[j])$;
  - Update $h_c \leftarrow h_c \oplus a[j]$.
- Return β.

FakeRefresh$(\mathsf{hint}, \mathsf{st}) \to \mathsf{hint}$.

- Parse $(x, j^*, c, q, r) \leftarrow \mathsf{st}$. Call $\mathsf{Hist.Append}(c)$.

(b) WR25 with Rewind technique

Figure 4: SinglePass [4] and WR25 [44]

single-query versions of SinglePass [4] and WR25 [44].

5 Keyword PIR

In this section, we propose our Keyword PIR scheme based on BPIR with KVS encoding. We note that Batch SI-PIR can then be seen as a BPIR instantiated using SI-PIR [4, 44, 46]. The KVS includes MPHF-KVS and BFF-KVS, which are respectively instantiated using MPHF [27, 29] and BFF [19].

5.1 Keyword PIR from BPIR and KVS

Our approach begins by encoding the key-value pairs KV into an indexable array DB via KVS encoding. Consequently, each keyword query corresponds to retrieving multiply (or one) specific indices from DB. We then apply BPIR to fetch these indices simultaneously in a batch query. Our Keyword PIR is defined in Figure 5.

$\mathsf{Setup}(1^\lambda, 1^\kappa, \mathsf{KV}) \to (\mathsf{DB}, \mathsf{hint}_s, \mathsf{hint}_c)$:

- The server invokes $\mathsf{KVS.Setup}(1^\lambda, \mathsf{KV}) \to \mathsf{pp}$, sends pp to client, runs $\mathsf{KVS.Encode}(\mathsf{pp}, \mathsf{KV}) \to (\mathcal{H}, \mathsf{hint}_{\mathsf{kvs}}, \mathsf{DB})$;
- The client invokes $\mathsf{BPIR.Setup}(1^\lambda, 1^\kappa, \mathsf{DB}) \to (\mathsf{hint}_s, \mathsf{hint}_c)$, sends hint_s to the server, and then erases the DB.

$\mathsf{Query}(\mathsf{hint}_c, k) \to (\mathsf{st}, q)$:

- Invoke $\mathsf{BPIR.Query}(\mathsf{hint}_c, \mathcal{H}(k)) \to (\mathsf{st}, q)$.

$\mathsf{Answer}(\mathsf{hint}_s, \mathsf{DB}, q) \to a$:

- Invoke $\mathsf{BPIR.Answer}(\mathsf{hint}_s, \mathsf{DB}, q) \to a$.

$\mathsf{Reconstruct}(\mathsf{hint}_c, \mathsf{st}, a) \to v$:

- Invoke $\mathsf{BPIR.Reconstruct}(\mathsf{hint}_c, \mathsf{st}, a) \to \beta$;
- Then invoke $\mathsf{KVS.Decode}(\mathsf{pp}, k, \mathsf{hint}_{\mathsf{kvs}}, \beta) \to v$.

Figure 5: The Keyword PIR form BPIR and KVS encoding

The correctness and security of our Keyword PIR are formally stated in Theorem 4.1.

Theorem 4.1 (Correctness and Security) Assume that BPIR satisfies correctness and security, and the KVS provides both inclusion and exclusion correctness. Then, the proposed Keyword PIR scheme satisfies correctness and security.

Proof. By the correctness of BPIR, we have that $\beta = \mathsf{DB}[\mathcal{H}(k)]$ holds with probability at least $1 - \mathsf{negl}(\lambda) - \mathsf{negl}(\kappa)$, while the inclusion and exclusion correctness of KVS ensure that if $k \in \mathsf{KV}$, $v = \mathsf{KV}[k]$ with probability at least $1 - \mathsf{negl}(\lambda) - \mathsf{negl}(\kappa)$; otherwise, if $k \notin \mathsf{KV}$, $v = \perp$ with probability at least $1 - 2^{-\mu_0}$. The security of Keyword PIR follows directly from the security of BPIR. □

Table 1: Performance of 3-sparse BFF-KVS. KV: key-value pairs, vlen: value length, flen: fingerprint length.

KV		3-sparse BFF-KVS encoding array		
Num	vlen (B)	flen (B)	DB size (MB)	time (s)
2^{25}	4	4	288	38.1
	28		1152	53.9
	60		2304	64.6

6 Implementation and Evaluation

In this section, we present the implementation details along with the corresponding evaluation results. The experiments were conducted on a machine equipped with an Intel(R) Xeon(R) Silver 4214R CPU @ 2.40GHz (32 cores, 128 GB RAM), running Ubuntu 22 and Go 1.23. All implementations were executed in a single-threaded setting.

6.1 Implementation Details of KVS

To handle large-scale keyword databases, we employ a two-level map structure, limiting each second-level map to a maximum of 32 million keywords. The first-level map encodes keys into indices and occupies no more than 4 KB. Therefore, it is sufficient to evaluate performance on 32 million keywords, while the remaining keywords can be handled similarly.

In BFF-KVS, we set the key length in KV to $\mu_1 = 64$ (e.g. 8 bytes) and the fingerprint length in the KVS to $\mu_0 = 32$ (e.g. 4 bytes), which can be configured to different lengths as needed. Following the constructions in [9, 19, 34], we instantiate the γ-sparse KVS using BFF [19] with $\gamma = 3$, by selecting appropriate parameters s and n as illustrated in Figure 6. The expansion rate is defined as $n = \rho \cdot m$, where $\rho \in [1.125, 1.15]$ in our evaluation. Table 1 presents the evaluation results for the 3-sparse KVS.

In the MPHF-KVS setting, we follow the default parameters and evaluate performance on a dataset of 32 million keywords. For BBHash, the encode time is 31.23 s and the decode time is 154.1 ns. The size of hint is 12.26 MB, corresponding to a space overhead of 3.065 bits per key. For Consensus RecSplit, we set the bucket size to 8192 and the space overhead parameter to 0.01. Under this configuration, the encode time is 168.5 s and the decode time is 243 ns. The resulting hint occupies 5.83 MB, achieving a space usage of 1.46 bits per key.

6.2 Evaluation Results

We evaluated our Keyword PIR: Piano$^{\text{bff-kvs}}$, SinglePass$^{\text{bff-kvs}}$ and WR25$^{\text{bff-kvs}}$ using three subroutines: Piano (Figure 3), SinglePass (Figure 4(a)) and WR25 (Figure 4(b)) within the Batch PIR framework. Since the Rewind and Skip versions of Piano$^{\text{kvs}}$ exhibit nearly identical performance, we only present the Skip version here. In MPHF-KVS, we implement the Piano$^{\text{mphf-kvs}}$, SinglePass$^{\text{mphf-kvs}}$, WR25$^{\text{mphf-kvs}}$, SimplePIR$^{\text{mphf-kvs}}$, DoublePIR$^{\text{mphf-kvs}}$. Since the encoding time of Consensus RecSplit is prohibitively long, we choose BBHash to instantiate our MPHF-KVS, trading a modest increase in memory usage for significantly faster encode. We also implemented DoublePIR [21] with KVS to realize Keyword PIR, enriching our benchmark. The results are then compared with the state-of-the-art KPIR [34] in Table 2.

The Piano$^{\text{bff-kvs}}$ supports a limited number of queries, which is sufficient for practical applications, but requires a longer offline phase. Its online communication cost is optimal, consisting of only $\sqrt{n}+1$ elements. While the client's online computation cost is slightly higher, around 40 ms, the server-side computation remains minimal, not exceeding 5 ms for a database of 1 billion entries. The SinglePass$^{\text{bff-kvs}}$ supports an unlimited number of queries but requires an additional update server. It has a shorter offline phase and minimal client hint storage, the online query and response times remain low, under 20 ms, with a communication cost of 1.7 MB for a database of 1 billion entries. However, during execution, the client's storage requirement grows to the size of the database. The WR25$^{\text{bff-kvs}}$ supports a limited number of queries depending on the configuration. It is a single-server PIR scheme with a shortest offline phase. Its online communication cost is $2\cdot\sqrt{n}$ elements, which is half that of SinglePass$^{\text{bff-kvs}}$. The reconstruct time is slightly higher, around 40 ms, while the online query and response require less than 13 ms and 1 MB, respectively, for a database of 1 billion entries. It is worth noting that all three schemes incur a substantial one-time communication cost during the setup phase. In contrast, KPIR$^{\text{bff-kvs}}$ [34] and DoublePIR$^{\text{bff-kvs}}$ handle 256 million entries and incur online costs of approximately 1.3 second and 800 KB.

In MPHF-KVS encoding schemes, the setup time and hint size increase linearly compared to those of BFF-KVS encoding schemes, since MPHF requires additional client-side storage, whereas BFF does not incur any client-side state. In contrast, the online computation and communication costs of MPHF-KVS encodings are lower than those of BFF-KVS and are almost identical to those of the corresponding Index PIR schemes.

Theoretically, our Batch PIR achieves performance comparable to that of Index PIR schemes [4, 44, 46]. Unlike approaches that execute multiple queries independently, Batch PIR completes all queries within a single round, thereby reducing network latency. When combined with a 3-sparse KVS, our Keyword PIR achieves online computation and communication costs within a constant factor

$$\frac{\mu_2+\mu_0}{\mu_2}\cdot\rho\cdot\gamma\leq 6.90 \tag{10}$$

where the fingerprint length is $\mu_0=4$ bytes and the value length is $\mu_2=4$ bytes.

For Keyword PIR schemes based on MPHF-KVS, a keyword query is completed with a single index query. The online computation and communication overhead consists of the Index PIR cost, the MPHF evaluation overhead, and the additional fingerprint storage in the database, resulting in a constant factor

$$\frac{\mu_2+\mu_0}{\mu_2} \tag{11}$$

which equals 2 when $\mu_0=4$ bytes, $\mu_2=4$ bytes. As $\mu_2\to\infty$, this constant factor converges to 1.

These results indicate that the online computation and communication costs of keyword queries incur less than a $7\times$ overhead for BFF-KVS with Batch PIR and a $2\times$ overhead for MPHF-KVS with Index PIR compared to the original Index PIR. Overall, our Keyword PIR schemes achieve overheads close to the their Index PIR, and preserving the underlying Index PIR performance metrics while scaling to databases with more than one billion entries.

7 Conclusion

In this work, we present a Keyword PIR scheme based on Batch PIR with KVS encoding. We propose a new MPHF-KVS encoding using a Minimal Perfect Hash Function (MPHF) [27, 29], which allows each keyword query to be resolved with a single index query. Consequently, the online computation and communication incur at most $2\times$ overhead compared to Index PIR.

Than, we propose a Batch PIR framework leverages the Rewind & Skip techniques combined with SI-PIR to realizes the Batch SI-PIR. After BFF-KVS encoding key-value pairs into an array that requires multiple index positions for access, the main challenge is to query these random indices in a single round. We address this by exploiting the independence between the query sets and the response elements. Then the Batch SI-PIR achieves comparable performance to the PIR schemes [4, 44, 46], while the Keyword PIR incurs on more than $7\times$ overhead of online computation and communication. Nonetheless, it still achieves substantial improvements in performance over state-of-the-art KPIR [34].

It is worth noting that the Rewind & Skip techniques constitute an independent area of research. Their primary function is to manage the program state at two different points in time. These techniques not only enable the conversion of multiple rounds into a single round but also reduce storage requirements by saving the input state rather than the final outputs.

Table 2: Performance of our scheme and KPIR$^{\text{bff-kvs}}$ [34], DoublePIR$^{\text{bff-kvs}}$, SimplePIR$^{\text{mphf-kvs}}$, DoublePIR$^{\text{mphf-kvs}}$,. In KV, the $2^{25} \times 4$ indicates that the key scale is 2^{25} and the value length is 4 bytes. QNum denotes the supported number of queries. The hint refers to the data the client needs to store.

Scheme	KV	Setup			Query		Answer	
	size(B)	QNum	time(s)	hint(MB)	time(ms)	com(KB)	time(ms)	com(KB)
Piano$^{\text{bff-kvs}}$	$2^{25} \times 4$	35730	1024	35.7	7.10	72.0	0.383	0.023
	$2^{28} \times 4$	113107	9370	113	20.0	203	1.29	0.023
	$2^{30} \times 4$	242274	44126	240	40.9	407	4.62	0.023
	$2^{25} \times 28$	35730	1094	67.2	7.17	72.0	0.875	0.094
	$2^{28} \times 28$	113107	10481	213	20.4	203	3.08	0.094
	$2^{25} \times 60$	35730	1197	109	7.06	72.0	1.34	0.188
SinglePass$^{\text{bff-kvs}}$	$2^{25} \times 4$	any	3.59	0.059	2.29	144	1.47	288
	$2^{28} \times 4$		20.1	0.149	7.67	407	5.73	814
	$2^{30} \times 4$		96.0	0.298	16.3	814	12.2	1629
	$2^{25} \times 28$	any	4.98	0.211	2.81	144	3.18	1152
	$2^{28} \times 28$		40.5	0.596	7.44	407	11.5	3258
	$2^{25} \times 60$	any	7.82	0.422	3.10	144	6.33	2304
WR25$^{\text{bff-kvs}}$	$2^{25} \times 4$	2048	1.75	0.106	1.71	72.0	0.729	144
	$2^{28} \times 4$	5792	11.8	0.298	4.73	203	2.63	407
	$2^{30} \times 4$	11585	46.5	0.596	12.2	407	6.63	814
	$2^{25} \times 28$	2048	1.76	0.422	2.35	72.0	1.94	576
	$2^{28} \times 28$	5792	15.3	1.19	6.09	204	5.51	1629
	$2^{25} \times 60$	2048	2.17	0.843	1.65	72.0	2.66	1152
KPIR$^{\text{bff-kvs}}$ [34]	$2^{25} \times 4$	any	121	67.9	121	204	142	203
	$2^{28} \times 4$		1087	192	336	576	1230	576
	$2^{25} \times 28$	any	458	129	233	1550	478	1551
DoublePIR$^{\text{bff-kvs}}$	$2^{25} \times 4$	any	145	128	456	775	161	768
	$2^{28} \times 4$		1189	128	492	822	1280	786
	$2^{25} \times 28$	any	349	464	427	3099	512	11141
Piano$^{\text{mphf-kvs}}$	$2^{25} \times 4$	100378	1480	55.1	2.2	32	0.233	0.008
	$2^{25} \times 28$	100378	1508	93.4	2.27	32	1.24	0.031
SinglePass$^{\text{mphf-kvs}}$	$2^{25} \times 4$	any	34.62	13.4	0.132	32	0.303	64
	$2^{28} \times 4$		280.8	106.1	1.51	128	1.54	256
	$2^{30} \times 4$		1320	424.5	3.96	256	4.04	512
	$2^{25} \times 28$	any	35.79	13.59	0.132	32	0.697	256
	$2^{28} \times 28$		286.2	106.3	1.53	128	3.11	1024
	$2^{25} \times 60$	any	37.6	13.82	0.132	32	1.12	512
WR25$^{\text{mphf-kvs}}$	$2^{25} \times 4$	8192	33.91	13.4	0.41	16	0.155	32
	$2^{28} \times 4$	16384	390.1	106.3	1.21	64	0.823	128
	$2^{30} \times 4$	32768	899.0	424.5	2.96	128	1.81	256
	$2^{25} \times 28$	8192	33.21	13.8	0.132	16.0	0.01	128
	$2^{28} \times 28$	16384	267.3	107.22	1.14	64	1.38	512
	$2^{25} \times 60$	2048	25.19	14.4	0.132	16.0	0.01	256
SimplePIR$^{\text{mphf-kvs}}$	$2^{25} \times 4$	any	145	81.2	35.4	64	50.3	64
	$2^{28} \times 4$		903	286	90.6	181	271	181
	$2^{25} \times 28$	any	610	183	85.2	170	231	171
DoublePIR$^{\text{mphf-kvs}}$	$2^{25} \times 4$	any	130	140.5	130	258	51	256
	$2^{28} \times 4$		936	233.1	125	272	356	256

References

[1] Davidson Alex, Pestana Gonçalo, and Celi Sofía. Frodopir: Simple, scalable, single-server private information retrieval. In *Proceedings on Privacy Enhancing Technologies*, volume 2023, pages 365–383, 2023. doi:10.56553/popets-2023-0022.

[2] Asra Ali, Tancrède Lepoint, Sarvar Patel, Mariana Raykova, Phillipp Schoppmann, Karn Seth, and Kevin Yeo. Communication–Computation trade-offs in PIR. In *Proceedings of the 30th USENIX Conference on Security Symposium*, SEC '21, pages 1811–1828. USENIX Association, 2021. URL: https://www.usenix.org/conference/usenixsecurity21/presentation/ali.

[3] Sebastian Angel, Hao Chen, Kim Laine, and Srinath Setty. Pir with compressed queries and amortized query processing. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 962–979, 2018. doi:10.1109/SP.2018.00062.

[4] Lazzaretti Arthur and Papamanthou Charalampos. Single pass client-preprocessing private information retrieval. In *Proceedings of the 33nd USENIX Conference on Security Symposium*, SEC '24, USA, 2024. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/lazzaretti.

[5] Amos Beimel, Yuval Ishai, and Tal Malkin. Reducing the servers' computation in privateinformation retrieval: Pir with preprocessing. *Journal of Cryptology*, 17(2):125–151, 2004. doi:10.1007/s00145-004-0134-y.

[6] Chor Benny, Gilboa Niv, and Naor Moni. Private information retrieval by keywords. *Cryptology ePrint Archive, Paper 1998/003*, 1998. URL: https://eprint.iacr.org/1998/003.

[7] Alexander Burton, Samir Jordan Menon, and David J. Wu. Respire: High-rate pir for databases with small records. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, pages 1463–1477, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3690328.

[8] Aguilar-Melchor Carlos, Barrier Joris, Fousse Laurent, and Killijian Marc-Olivier. Xpir: Private information retrieval for everyone. In *Proceedings on Privacy Enhancing Technologies*, pages 155–174, 2016. doi:10.1515/popets-2016-0010.

[9] Sofia Celi and Alex Davidson. Call me by my name: Simple, practical private information retrieval for keyword queries. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, pages 4107–4121, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3670271.

[10] Benny Chor, Eyal Kushilevitz, Oded Goldreich, and Madhu Sudan. Private information retrieval. *Journal of the ACM*, 45(6):965–981, 1998. doi:10.1145/293347.293350.

[11] Henry Corrigan-Gibbs, Alexandra Henzinger, and Dmitry Kogan. Single-server private information retrieval with sublinear amortized time. In *Advances in Cryptology – EUROCRYPT 2022*, pages 3–33, Cham, 2022. Springer International Publishing. doi:10.1007/978-3-031-07085-3_1.

[12] Henry Corrigan-Gibbs and Dmitry Kogan. Private information retrieval with sublinear online time. In *Advances in Cryptology – EUROCRYPT 2020*, pages 44–75, Cham, 2020. Springer International Publishing. doi:10.1007/978-3-030-45721-1_3.

[13] Marian Dietz and Stefano Tessaro. Fully malicious authenticated pir. In *Advances in Cryptology – CRYPTO 2024*, pages 113–147, Cham, 2024. Springer Nature Switzerland. doi:10.1007/978-3-031-68400-5_4.

[14] Kogan Dmitry and Corrigan-Gibbs Henry. Private blocklist lookups with checklist. In *roceedings of the 30th USENIX Conference on Security Symposium*, SEC '21, pages 875–892. USENIX Association, 2021. URL: https://www.usenix.org/conference/usenixsecurity21/presentation/kogan.

[15] Nico Döttling, Jesko Dujmovic, Julian Loss, and Maciej Obremski. Minicrypt PIR for big batches. *Cryptology ePrint Archive, Paper 2025/317*, 2025. URL: https://eprint.iacr.org/2025/317.

[16] Emmanuel Esposito, Thomas Mueller Graf, and Sebastiano Vigna. Recsplit: Minimal perfect hashing via recursive splitting. In *2020 Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*, pages 175–185. URL: https://epubs.siam.org/doi/abs/10.1137/1.9781611976007.14.

[17] Bin Fan, Dave G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. Cuckoo filter: Practically better than bloom. In *Proceedings of the 10th ACM International on Conference on Emerging Networking Experiments and Technologies*, CoNEXT '14, pages 75–88, New York, NY, USA, 2014. Association for Computing Machinery. doi:10.1145/2674005.2674994.

[18] Adrià Gascón, Yuval Ishai, Mahimna Kelkar, Baiyu Li, Yiping Ma, and Mariana Raykova. Computationally secure aggregation and private information retrieval in the shuffle model. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, pages 4122–4136, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3670391.

[19] Thomas Mueller Graf and Daniel Lemire. Binary fuse filters: Fast and smaller than xor filters. *ACM J. Exp. Algorithmics*, 27, 2022. doi:10.1145/3510449.

[20] Matthew Green, Watson Ladd, and Ian Miers. A protocol for privately reporting ad impressions at scale. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, pages 1591–1601, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/2976749.2978407.

[21] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: simple and fast single-server private information retrieval. In *Proceedings of the 32nd USENIX Conference on Security Symposium*, SEC '23, USA, 2023. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/henzinger.

[22] Swanand Kadhe, Brenden Garcia, Anoosheh Heidarzadeh, Salim El Rouayheb, and Alex Sprintson. Private information retrieval with side information. *IEEE Transactions on Information Theory*, 66(4):2032–2043, 2020. doi:10.1109/TIT.2019.2948845.

[23] Danie Kales, Christian Rechberger, Thomas Schneider, Matthias Senker, and Christian Weinert. Mobile private contact discovery at scale. In *Proceedings of the 28th USENIX Conference on Security Symposium*, SEC'19, pages 1447–1464, USA, 2019. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity19/presentation/kales.

[24] E. Kushilevitz and R. Ostrovsky. Replication is not needed: single database, computationally-private information retrieval. In *Proceedings 38th Annual Symposium on Foundations of Computer Science*, pages 364–373, 1997. doi:10.1109/SFCS.1997.646125.

[25] Hans-Peter Lehmann, Thomas Mueller, Rasmus Pagh, Giulio Ermanno Pibiri, Peter Sanders, Sebastiano Vigna, and Stefan Walzer. Modern minimal perfect hashing: A survey. 2025. URL: https://arxiv.org/abs/2506.06536.

[26] Hans-Peter Lehmann, Peter Sanders, and Stefan Walzer. Sichash - small irregular cuckoo tables for perfect hashing. pages 176–189. URL: https://epubs.siam.org/doi/abs/10.1137/1.9781611977561.ch15.

[27] Hans-Peter Lehmann, Peter Sanders, Stefan Walzer, and Jonatan Ziegler. Combined search and encoding for seeds, with an application to minimal perfect hashing. 2025. URL: https://arxiv.org/abs/2502.05613.

[28] Baiyu Li, Daniele Micciancio, Mariana Raykova, and Mark Schultz-Wu. Hintless single-server private information retrieval. In *Advances in Cryptology – CRYPTO 2024*, pages 183–217, Cham, 2024. Springer Nature Switzerland. doi:10.1007/978-3-031-68400-5_6.

[29] Antoine Limasset, Guillaume Rizk, Rayan Chikhi, and Pierre Peterlongo. Fast and scalable minimal perfect hashing for massive key sets. *arXiv:1702.03154*, 2017. URL: https://arxiv.org/abs/1702.03154.

[30] Yehuda Lindell. How to simulate it - a tutorial on the simulation proof technique. *Cryptology ePrint Archive, Paper 2016/046*, 2016. URL: https://eprint.iacr.org/2016/046.

[31] Ming Luo, Feng-Hao Liu, and Han Wang. Faster fhe-based single-server private information retrieval. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, pages 1405–1419, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3690233.

[32] Rasoul Akhavan Mahdavi and Florian Kerschbaum. Constant-weight PIR: Single-round keyword PIR via constant-weight equality operators. In *Proceedings of the 31st USENIX Conference on Security Symposium*, SEC '22, pages 1723–1740, Boston, MA, 2022. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity22/presentation/mahdavi.

[33] Rasoul Akhavan Mahdavi, Sarvar Patel, Joon Young Seo, and Kevin Yeo. InsPIRe: Communication-efficient PIR with silent preprocessing. *Cryptology ePrint Archive, Paper 2025/1352*, 2025. URL: https://eprint.iacr.org/2025/1352.

[34] Hao Meng, Liu Weiran, Peng Liqiang, Zhang Cong, Wu Pengfei, Zhang Lei, Li Hongwei, and Deng Robert H. Practical keyword private information retrieval from key-to-index mappings. In *Proceedings of the 34nd USENIX Conference on Security Symposium*, SEC '25, USA, 2025. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity25/presentation/hao.

[35] Samir Jordan Menon and David J. Wu. Spiral: Fast, high-rate single-server pir via fhe composition. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 930–947, 2022. doi:10.1109/SP46214.2022.9833700.

[36] Samir Jordan Menon and David J. Wu. Ypir: high-throughput single-server pir with silent preprocessing. In *Proceedings of the 33rd USENIX Conference on Security Symposium*, SEC '24, USA, 2024. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity24/presentation/menon.

[37] Muhammad Haris Mughees, Hao Chen, and Ling Ren. Onionpir: Response efficient single-server pir. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, CCS '21, pages 2292–2306, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3460120.3485381.

[38] Sarvar Patel, Giuseppe Persiano, and Kevin Yeo. Private stateful information retrieval. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, CCS '18, pages 1002–1019, New York, NY, USA, 2018. Association for Computing Machinery. doi:10.1145/3243734.3243821.

[39] Sarvar Patel, Joon Young Seo, and Kevin Yeo. Don't be dense: efficient keyword pir for sparse databases. In *Proceedings of the 32nd USENIX Conference on Security Symposium*, SEC '23, USA, 2023. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity23/presentation/patel.

[40] Luciano Patrão. *Virtual Machine Snapshots*, pages 419–446. Apress, Berkeley, CA, 2024. doi:10.1007/979-8-8688-0208-9_22.

[41] Giulio Ermanno Pibiri and Roberto Trani. Pthash: Revisiting fch minimal perfect hashing. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '21, pages 1339–1348, New York, NY, USA, 2021. Association for Computing Machinery. URL: https://doi.org/10.1145/3404835.3462849.

[42] Lehmkuhl Ryan, Henzinger Alexandra, and Corrigan-Gibbs Henry. Distributional private information retrieval. In *Proceedings of the 34nd USENIX Conference on Security Symposium*, SEC '25, USA, 2025. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity25/presentation/lehmkuhl.

[43] Kurt Thomas, Jennifer Pullman, Kevin Yeo, Ananth Raghunathan, Patrick Gage Kelley, Luca Invernizzi, Borbala Benko, Tadek Pietraszek, Sarvar Patel, Dan Boneh, and Elie Bursztein. Protecting accounts from credential stuffing with password breach alerting. In *Proceedings of the 28th USENIX Conference on Security Symposium*, SEC '19, pages 1556–1571, Santa Clara, CA, 2019. USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity19/presentation/thomas.

[44] Zhikun Wang and Ling Ren. Single-server client preprocessing pir with tight space-time trade-off. In *Advances in Cryptology – EUROCRYPT 2025*, pages 94–122, Cham, 2025. Springer Nature Switzerland. doi:10.1007/978-3-031-91095-1_4.

[45] Kevin Yeo. Lower bounds for (batch) pir with private preprocessing. In *Advances in Cryptology – EUROCRYPT 2023*, pages 518–550, Cham, 2023. Springer Nature Switzerland. doi:10.1007/978-3-031-30545-0_18.

[46] Mingxun Zhou, Andrew Park, Wenting Zheng, and Elaine Shi. Piano: Extremely simple, single-server pir with sublinear server computation. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4296–4314, 2024. doi:10.1109/SP54263.2024.00055.

8 Appendix

Here, we provide the relocation algorithm of WR25 [44] in Figure 6, and present the details of the KVS algorithm based on the Binary Fuse Filter (BFF) in Figure 7.

Construction 3.1 (Relocation History). The data structure Hist holds an array C of consumed positions and a map M linking each position to its index in C. It supports:

- $\mathsf{Hist.Init}()$: Clears C and M to an empty array and hash map.
- $\mathsf{Hist.Append}(c)$: Append c to C and sets $M[c] = |C| - 1$.
- $\mathsf{Hist}[t] \to c$: Return $C[t]$ if $t < |C|$, else $\perp$.
- $\mathsf{Hist}^{-1}[c] \to t$: Return $M[c]$ if $c \in M$, else $\perp$.

Construction 3.2 (Relocation Data Structure). Parameterized by m, m', with state st containing the permutation $P = (P_0, P_1, \dots)$ and Hist.

DS.Init(sk, T, m'):

- For each $j \in [T]$, set $P_j = \mathsf{PRP}(\mathsf{sk}_j, \cdot)$ over $[m']$, where $\mathsf{sk}_j = \mathsf{PRF}(\mathsf{sk}, j)$.
- Invoke $\mathsf{Hist.Init}()$.

DS.Access$(i, c) \to e$:

- While $\mathsf{Hist}[P_i^{-1}(c) - m] \neq \perp$:
  - update $c \leftarrow \mathsf{Hist}[P_i^{-1}(c) - m]$.
- Return $P_i^{-1}(c)$ if $P_i^{-1}(c) < m$, else $\perp$.

DS.Locate$(i, e) \to c$:

- Set $c = P_i(e)$.
- While $\mathsf{Hist}^{-1}[c] \neq \perp$:
  - update $c \leftarrow P_i(m + \mathsf{Hist}^{-1}[c])$.
- Return c.

DS.Relocate(c):

- Invoke $\mathsf{Hist.Append}(c)$.

Figure 6: The relocation algorithm of WR25 [44]

Construction 4: 3-Sparse KVS from 3-wise Binary Fuse Filters. Parameterized by $\mu_0 \in \{8, 16, 32\}$ as the fingerprint length, $\mu_1 = \log_2 |\mathcal{K}|$, and $\mu_2 = \log_2 |\mathcal{V}|$. The key-value pairs are defined as $\mathsf{KV} \in (\mathcal{K} \times \mathcal{V})^m$.

Setup$(\varepsilon, m) \to \mathsf{pp}$.

- Initialize: $s = 2^{\lceil \log_{3.33} m + 2.25 \rceil} \approx 4.8 \cdot m^{0.58}$, $n = \max\left(\left\lfloor \left(0.875 + 0.25 \cdot \max\left(1, \frac{\log 10^6}{\log m}\right)\right) \cdot m \right\rfloor, \lfloor 1.125m \rfloor\right)$.
- Define the fingerprint function as $\mathsf{fpt}_\varepsilon(k, v) = \mathsf{hash}(k) \| v$, where $\mathsf{hash} : \{0,1\}^* \mapsto \{0,1\}^{\mu_0}$, with $\mu_0 = \log_2(\varepsilon^{-1})$.
- Return $\mathsf{pp} = (s, m, \mathsf{fpt}_\varepsilon(\cdot,\cdot))$.

Encode$(\mathsf{pp}, \mathsf{KV}) \to (\mathcal{H}, \mathsf{DB})$.

- Let S be the set of m distinct keys in KV. Allocate an empty array $\mathsf{DB} \in Z^n_{\mu_0+\mu_2}$, partitioned into segments of size s.
- Repeat:
  - Choose hash functions h_0, h_1, h_2 where each $h_i : \mathcal{K} \mapsto [n]$, ensuring that $h_0(k)$, $h_1(k)$, and $h_2(k)$ map into three distinct and consecutive segments. Specifically, let $h_i(k) = (h'(k) + i) \cdot s + h''(k \| i)$, with $h' : \mathcal{K} \mapsto [n/s - 2]$, and $h'' : \mathcal{K} \mapsto [s]$.
  - Sort all keys $k \in S$ based on the segment index of $h_0(k)$.
  - Allocate an empty array c of size m. For each $k \in S$, add k to the sets $c[h_0(k)]$, $c[h_1(k)]$, and $c[h_2(k)]$.
  - Sequentially scan the array c. Whenever a singleton (a location with exactly one key) is found, push that location to a stack Q.
  - Initialize an empty stack P.
    - While stack Q is non-empty:
      - Pop a location i from Q.
      - If $c[i]$ is a singleton:
        Push the key k in $c[i]$ and location i into stack P.
        Remove k from $c[h_0(k)]$, $c[h_1(k)]$, and $c[h_2(k)]$.
        If any of those sets become singleton, push their location to Q.
- Until the size of P equals m.
  - While stack P is non-empty:
    - Pop a key k and location i from P, where $i \in \{h_0(k), h_1(k), h_2(k)\}$.
    - Set $\mathsf{DB}[i] \leftarrow \mathsf{DB}[h_0(k)] \oplus \mathsf{DB}[h_1(k)] \oplus \mathsf{DB}[h_2(k)] \oplus \mathsf{fpt}_\varepsilon(k, \mathsf{KV}[k])$.
- Return $(\mathcal{H} = \{h_0, h_1, h_2\}, \mathsf{DB})$.

Decode$(k, \{x_0, x_1, x_2\}, \mathsf{fpt}_\varepsilon(\cdot,\cdot)) \to v$.

- Compute $v_1 \| v_2 \leftarrow x_0 \oplus x_1 \oplus x_2$ using the same fingerprint function.
- If v_1 is equal to $\mathsf{hash}(k)$, then set $v \leftarrow v_2$; otherwise, $v \leftarrow \perp$.
- Return v.

Figure 7: 3-Sparse KVS Construction Using 3-wise Binary Fuse Filters